



Using Multidimensional Storage (Globals)

Version 2026.1
2026-04-20

Using Multidimensional Storage (Globals)
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1
Copyright © 2026 InterSystems Corporation
All rights reserved.

InterSystems®, HealthShare Care Community®, HealthShare Unified Care Record®, InterSystems IRIS® IntegratedML®, InterSystems Caché®, InterSystems Ensemble®, InterSystems HealthShare®, InterSystems IRIS®, InterSystems IRIS® for Health, and TrakCare are registered trademarks of InterSystems Corporation. HealthShare® Health Connect Cloud™, InterSystems® Data Fabric Studio™, and InterSystems Supply Chain Orchestrator™ are trademarks of InterSystems Corporation. TrakCare is a registered trademark in Australia and the European Union.

All other brand or product names used herein are trademarks or registered trademarks of their respective companies or organizations.

This document contains trade secret and confidential information which is the property of InterSystems Corporation, One Congress Street, Boston, MA 02114, or its affiliates, and is furnished for the sole purpose of the operation and maintenance of the products of InterSystems Corporation. No part of this publication is to be used for any other purpose, and this publication is not to be reproduced, copied, disclosed, transmitted, stored in a retrieval system or translated into any human or computer language, in any form, by any means, in whole or in part, without the express prior written consent of InterSystems Corporation.

The copying, use and disposition of this document and the software programs described herein is prohibited except to the limited extent set forth in the standard software license agreement(s) of InterSystems Corporation covering such programs and related documentation. InterSystems Corporation makes no representations and warranties concerning such software programs other than those set forth in such standard software license agreement(s). In addition, the liability of InterSystems Corporation for any losses or damages relating to or arising out of the use of such software programs is limited in the manner set forth in such standard software license agreement(s).

THE FOREGOING IS A GENERAL SUMMARY OF THE RESTRICTIONS AND LIMITATIONS IMPOSED BY INTERSYSTEMS CORPORATION ON THE USE OF, AND LIABILITY ARISING FROM, ITS COMPUTER SOFTWARE. FOR COMPLETE INFORMATION REFERENCE SHOULD BE MADE TO THE STANDARD SOFTWARE LICENSE AGREEMENT(S) OF INTERSYSTEMS CORPORATION, COPIES OF WHICH WILL BE MADE AVAILABLE UPON REQUEST.

InterSystems Corporation disclaims responsibility for errors which may appear in this document, and it reserves the right, in its sole discretion and without notice, to make substitutions and modifications in the products and practices described in this document.

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

1 Introduction to Globals	1
1.1 What Are Globals?	1
1.2 Why Should Application Developers Learn About Globals?	2
1.3 Examples of Globals	2
1.3.1 Scalars	2
1.3.2 Arrays	2
1.3.3 Dictionaries	3
1.3.4 Ordered Trees	3
1.4 Globals and External Languages	3
2 Formal Rules about Globals	5
2.1 Introduction to Global Names and Limits	5
2.1.1 Variations	6
2.2 Introduction to Global Nodes and Subscripts	6
2.3 Rules for Global Subscripts	6
2.4 Collation of Globals	7
2.5 See Also	7
3 Global Mapping and Subscript-Level Mapping	9
3.1 Simple Example of Subscript-Level Mapping	9
3.2 More Complex Example of Subscript-Level Mapping	10
3.3 Key Principles	10
3.3.1 Using Distinct Ranges of Globals and Subscripts	10
3.3.2 Logging Changes	10
3.4 See Also	11
4 SQL and Persistent Class Use of Multidimensional Storage	13
4.1 Storage Definitions	13
4.1.1 Default Structure	13
4.1.2 IDKEY	14
4.1.3 Subclasses	15
4.1.4 Parent-Child Relationships	16
4.1.5 Embedded Objects	16
4.1.6 Streams	17
4.2 Indices	17
4.2.1 Storage Structure of Standard Indexes	17
4.3 Bitmap Indexes	18
4.3.1 Logical Operation of Bitmap Indexes	18
4.3.2 Storage Structure of Bitmap Indexes	19
4.3.3 Direct Access of Bitmap Indexes	20
4.4 See Also	20
5 Temporary Globals and the IRISTEMP Database	21
5.1 Using Temporary Globals	21
5.2 Defining a Mapping for Temporary Globals	22
5.3 System Use of IRISTEMP	23
5.4 ^CacheTemp Globals	23
5.5 See Also	23

1

Introduction to Globals

This page introduces *globals*, the underlying multidimensional storage structure for InterSystems IRIS® data platform. No matter how you decide to store or access your data, what you're doing is using globals.

Globals can be accessed using a relational model, using an object model, or directly. For a video that explains the benefits of this multi-model access and a hands-on exercise that lets you try the three alternatives yourself, see [Exploring Multiple Data Models with Globals](#).

1.1 What Are Globals?

One of the hallmarks of the InterSystems IRIS is its ability to store data once and allow you to access it using multiple paradigms. For example, you can use InterSystems SQL to visualize your data as rows and columns, or you can use ObjectScript and think of your data in terms of objects that have properties and methods. Your application can even mix these data models, using whichever model is easiest and more efficient for a given task. But no matter how you write or access your data, InterSystems IRIS stores it in underlying data structures known as globals.

Globals are persistent multidimensional sparse arrays:

- *Persistent* — Globals are stored in the database and can be retrieved at any time, by any process that can access that database.
- *Multidimensional* — The nodes in a global can have any number of subscripts. These subscripts can be integers, decimal numbers, or strings.
- *Sparse* — Node subscripts do not have to be contiguous, meaning that subscripts without a stored value do not use any storage.

Nodes in a global can store many types of data, including:

- Strings
- Numeric data
- Streams of character or binary data
- Collections of data, such as lists or arrays
- References to other storage locations

Even the server-side code you write is ultimately stored in globals!

1.2 Why Should Application Developers Learn About Globals?

While it is possible to write an application on the InterSystems IRIS platform with little or no knowledge of globals, there are several reasons why you may want to learn more about them:

- Some operations may be easier or more efficient if you access globals directly.
- You may want to create custom data structures for data that does not conform to relational or object data models.
- Some system administration tasks are done at the global level, and understanding globals will make these tasks more meaningful to you.

1.3 Examples of Globals

If you're new to InterSystems IRIS, you may be tempted to compare globals to data structures you have encountered from programming in other languages. This is a difficult exercise because a global is a flexible data structure that can be used in many different ways. But no matter the type of data it holds, a global is differentiated from a regular variable by placing a caret (^) in front of the name. This indicates that the variable is persisted to the database.

1.3.1 Scalars

In its simplest form, a global can be used to store a single value, or scalar:

```
^a = 4
```

In this example, the global ^a holds an integer with the value 4, but as mentioned earlier, it can hold data of other types just as easily.

1.3.2 Arrays

Globals can also be used as you would use an array in other languages, for example:

```
^month(1) = "January"  
^month(2) = "February"  
^month(3) = "March"  
^month(4) = "April"  
.  
.  
.  
^month(12) = "December"
```

However, not every subscript in the array must include data. Since an array can be sparse, no storage is allocated for locations in an array that are not used.

```
^sparse(1,2,3) = 16  
^sparse(1,2,5000) = 400
```

And, unlike arrays in many languages, the subscripts of a global can be negative numbers, real numbers, or strings. And the same array can hold data of varying types.

```
^misc(-4, "hello", 3.14) = 0
^misc("Sam", 27) = "Persimmon"
```

1.3.3 Dictionaries

Because of their flexibility, many people conceptualize globals as dictionaries (or nested dictionaries), with key-value pairs. In the following example, the global `^team` stores information about a baseball team:

```
^team("ballpark") = "Fenway Park"
^team("division") = "East"
^team("established") = 1901
^team("league") = "American"
^team("name") = "Boston Red Sox"
^team("retired number",1) = "Bobby Doerr"
^team("retired number",4) = "Joe Cronin"
^team("retired number",6) = "Johnny Pesky"
^team("retired number",8) = "Carl Yastrzemski"
^team("retired number",9) = "Ted Williams"
^team("world series titles") = $lb(1903,1912,1915,1916,1918,2004,2007,2013,2018)
```

In many languages, dictionaries are unordered, meaning that when you retrieve data from the dictionary, the data can be returned in some unspecified order. With globals, however, data is sorted according to its subscripts as it is stored.

1.3.4 Ordered Trees

It is more accurate to visualize a global as an ordered tree, where each node in the tree can have a value and/or children. In this regard, it is more flexible than nested dictionaries in other languages, where typically only the leaves of the tree contain data. In the following example, the global `^bird` stores birds according to their scientific names, with the names of each bird stored at the leaves of the tree. Here, the root node stores an overall description of the global, while a node representing a family of birds stores a description of that family:

```
^bird = "Birds of North America"
^bird("Anatidae") = "Ducks, Geese and Swans"
^bird("Anatidae", "Aix", "sponsa") = "Wood Duck"
^bird("Anatidae", "Anas", "rubripes") = "American Black Duck"
^bird("Anatidae", "Branta", "leucopsis") = "Barnacle Goose"
^bird("Odontophoridae") = "New World Quails"
^bird("Odontophoridae", "Callipepia", "californica") = "California Quail"
^bird("Odontophoridae", "Callipepia", "gambelii") = "Gambel's Quail"
```

For an animated illustration of how data is stored in ordered trees, see [Ordered Trees](#).

1.4 Globals and External Languages

If you are writing an application in any of the supported external languages, InterSystems IRIS provides APIs that allow you to manipulate your data using the three models discussed in this topic, as follows:

- Relational access through JDBC, ADO.NET, DB-API, or ODBC
- Object access through the InterSystems XEP APIs for Java and .Net
- Direct access to globals through the InterSystems Native SDKs

Note: Not all forms of access are supported for all languages.

2

Formal Rules about Globals

This page describes formal rules governing [globals](#) and global references (apart from [extended global references](#), discussed separately).

Note: This page has some examples that use ObjectScript. Except where noted, this page applies to any language.

2.1 Introduction to Global Names and Limits

The basic rules for global names are as follows:

- The name begins with a caret character (^) prefix. This caret distinguishes a global from a local variable.
- The next character can be a letter or the percent character (%):
 - Globals with names that start ^% are available in all namespaces. These are sometimes called *percent globals*.
 - Globals with names that do not use % are available only in the current namespace unless there are global mappings in effect.
- The other characters of a global name may be letters, numbers, or the period (.) character, except that the last character of the name cannot be a period.
- A global name may be up to 31 characters long (exclusive of the caret character prefix). You can specify global names that are significantly longer, but InterSystems IRIS treats only the first 31 characters as significant.
- Global names are case-sensitive.
- InterSystems IRIS provides special treatment for globals with names that start ^IRIS.TempUser — for example, ^IRIS.TempUser.MyApp. If you create such globals, these globals are written to the IRISTEMP database; see [Temporary Globals and the IRISTEMP Database](#).
- There are naming conventions to follow to avoid collision with InterSystems globals; see [Global Variable Names to Avoid](#).
- InterSystems IRIS imposes a limit on the total length of a global reference, and this limit, in turn, imposes limits on the length of any subscript values. See [Maximum Length of a Global Reference](#).

For more details, see [Rules and Guidelines for Identifiers](#).

2.1.1 Variations

There are two variations that apply to code written in ObjectScript:

- An ObjectScript process-private global is an array variable that is only accessible to the process that created it. The name of a process-private global starts with ^| | rather than a single caret (^). For details, see [Process-Private Globals](#).
- In ObjectScript, you can refer to a global in another namespace via an [extended global reference](#).

2.2 Introduction to Global Nodes and Subscripts

A global typically has multiple *nodes*, generally identified by a *subscript* or set of subscripts. For a basic example:

ObjectScript

```
set ^Demo(1)="Cleopatra"
```

This statement refers to the global node ^Demo(1), which is a node within the ^Demo global. This node is identified by one subscript.

For another example:

ObjectScript

```
set ^Demo("subscript1", "subscript2", "subscript3")=12
```

This statement refers to the global node ^Demo("subscript1", "subscript2", "subscript3"), which is another node within the same global. This node is identified by three subscripts.

For yet another example:

ObjectScript

```
set ^Demo="hello world"
```

This statement refers to the global node ^Demo, which does not use any subscripts.

The nodes of a global form a hierarchical structure. ObjectScript provides commands that take advantage of this structure. You can, for example, remove a node or remove a node and all its children; see [Using Globals](#).

Important: Note that any global node cannot contain a string longer than the string length limit, which is extremely long. See [General System Limits](#).

2.3 Rules for Global Subscripts

Subscripts have the following rules:

- Subscript values are case-sensitive.
- A subscript value can be any ObjectScript expression, provided that the expression does not evaluate to the null string ("").

The value can include characters of all types, including blank spaces, non-printing characters, and Unicode characters. (Note that non-printing characters are less practical in subscript values.)

- Before resolving a global reference, InterSystems IRIS evaluates each subscript in the same way it evaluates any other expression. In the following example, we set one node of the ^Demo global, and then we refer to that node in several equivalent ways:

```
SAMPLES>s ^Demo(1+2+3)="a value"
```

```
SAMPLES>w ^Demo(3+3)
a value
```

```
SAMPLES>w ^Demo(03+03)
a value
```

```
SAMPLES>w ^Demo(03.0+03.0)
a value
```

```
SAMPLES>set x=6
```

```
SAMPLES>w ^Demo(x)
a value
```

- InterSystems IRIS imposes a limit on the total length of a global reference, and this limit, in turn, imposes limits on the length of any subscript values. See [Maximum Length of a Global Reference](#).

CAUTION: The preceding rules apply for all [InterSystems IRIS supported collations](#). For older collations still in use for compatibility reasons, such as “pre-ISM-6.1”, the rules for subscripts are more restrictive. For example, character subscripts cannot have a control character as their initial character; and there are limitations on the number of digits that can be used in integer subscripts.

2.4 Collation of Globals

Within a global, nodes are stored in a collated (sorted) order.

Applications typically control the order in which nodes are sorted by applying a conversion to values used as subscripts. For example, the SQL engine, when creating an index on string values, converts all string values to uppercase letters and prepends a space character to make sure that the index is both not case-sensitive and collates as text (even if numeric values are stored as strings).

2.5 See Also

- [Introduction to Globals](#)
- [Extended References](#)

3

Global Mapping and Subscript-Level Mapping

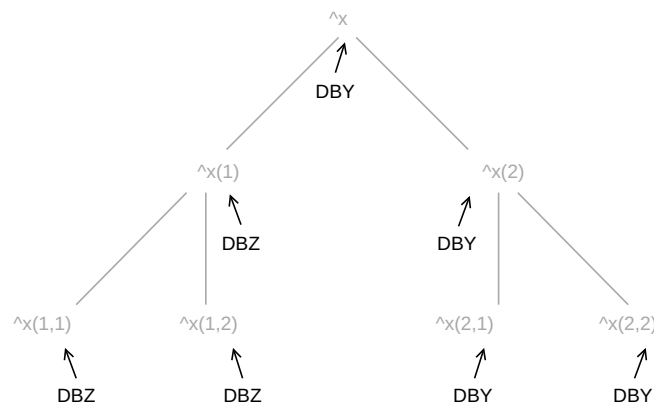
You can map [globals](#) and routines from one database to another on the same or different systems. This allows simple references to data which can exist anywhere and is the primary feature of a namespace. You can map whole globals or pieces of globals; mapping a piece of a global (or a subscript) is known as *subscript-level mapping (SLM)*.

You can map globals and routines from one database to another on the same or different systems. Because you can map global subscripts, data can easily span disks.

To configure this type of mapping, see [Adding Mappings to a Namespace](#).

3.1 Simple Example of Subscript-Level Mapping

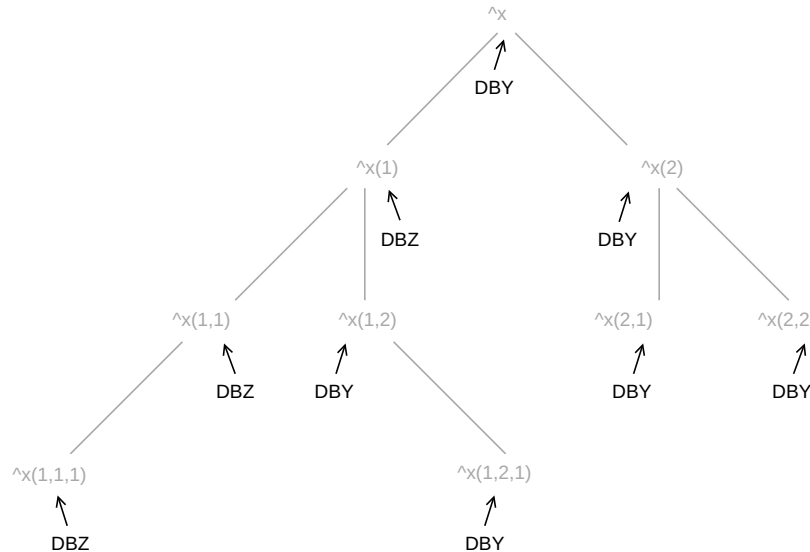
Global mapping is applied hierarchically. For example, if the NSX namespace has an associated DBX database, but maps the $\wedge x$ global to the DBY database and $\wedge x(1)$ to the DBZ database, then any subscripted form of the $\wedge x$ global — except those that are part of the $\wedge x(1)$ hierarchy — is mapped to DBY; those globals that are part of the $\wedge x(1)$ hierarchy are mapped to DBZ. The following diagram illustrates this hierarchy:



In this diagram, the globals and their hierarchy appear in gray, and the databases to which they are mapped appear in black.

3.2 More Complex Example of Subscript-Level Mapping

It is also possible to map part of a mapped, subscripted global to another database, or even back to the database to which the initial global is mapped. Suppose that the previous example had the additional mapping of the $\wedge x(1,2)$ global back to the DBY database. This would appear as follows:



Again, the globals and their hierarchy appear in gray, and the databases to which they are mapped appear in black.

Once you have mapped a global from one namespace to another, you can reference the mapped global as if it were in the current namespace — with a simple reference, such as $\wedge ORDER$ or $\wedge X(1)$.

Important: When establishing subscript-level mapping ranges, the behavior of string subscripts differs from that of integer subscripts. For strings, the first character determines the range, while the range for integers uses numeric values. For example, a subscript range of ("A"):("C") contains not only AA but also AC and ABCDEF; by contrast, a subscript range of (1):(2) does not contain 11.

3.3 Key Principles

3.3.1 Using Distinct Ranges of Globals and Subscripts

Each of a namespace's mappings must refer to distinct ranges of globals or subscripts. Mapping validation prevents the establishment of any kind of overlap. For example, if you attempt to use the Management Portal to create a new mapping that overlaps with an existing mapping, the Portal prevents this from occurring and displays an error message.

3.3.2 Logging Changes

Successful changes to the mappings through the Portal are also logged in `messages.log`; unsuccessful changes are not logged. Any failed attempts to establish mappings by hand-editing the configuration parameter (CPF) file are logged in `messages.log`; for details on editing the CPF, see [Editing the Active CPF](#).

3.4 See Also

- [Introduction to Globals](#)
- [Adding Mappings to a Namespace](#)

4

SQL and Persistent Class Use of Multidimensional Storage

This page describes how InterSystems IRIS® data platform persistent classes and SQL engine make use of multidimensional storage ([globals](#)) for storing persistent objects, relational tables, and indexes.

Though the InterSystems IRIS object and SQL engines automatically provide and manage data storage structures, it can be useful to understand the details of how this works.

The storage structures used by the object and relational view of data are identical. For simplicity, this document only describes storage from the object perspective.

4.1 Storage Definitions

Every persistent class that uses the `%Storage.Persistent` storage class (the default) can store instances of itself within the InterSystems IRIS database using one or more nodes of multidimensional storage (globals). Specifically, every persistent class has a *storage definition* that defines how its properties are stored within global nodes. This storage definition (referred to as “default structure”) is managed automatically by the class compiler. (You can modify this storage definition or even provide alternate versions of it if you like. This is not discussed in this document.)

4.1.1 Default Structure

The default structure used for storing persistent objects is quite simple:

- Data is stored in a global whose name starts with the complete class name, including package name. A **D** is appended to form the name of the data global, while an **I** is appended for the index global.
(See [Hashed Global Names](#) for an option that results in a shorter global name.)
- Data for each instance is stored within a single node of the data global with all non-transient properties placed within a **\$List** structure.
- Each node in the data global is subscripted by object ID value. For persistent classes (other than those created via SQL), the default storage structure uses **\$Increment** to assign unique object (row) identifier values. For persistent classes created via SQL, the default storage structure instead uses **\$Sequence**.

For example, suppose we define a simple persistent class, `MyApp.Person`, with two literal properties:

Class Definition

```
Class MyApp.Person Extends %Persistent
{
Property Name As %String;
Property Age As %Integer;
}
```

If we create and save two instances of this class, the resulting global will be similar to:

```
^MyApp.PersonD = 2 // counter node
^MyApp.PersonD(1) = $LB("",530,"Abraham")
^MyApp.PersonD(2) = $LB("",680,"Philip")
```

Note that the first piece of the **\$List** structure stored in each node is empty; this is reserved for a class name. If we define any subclasses of this Person class, this slot contains the subclass name. The **%OpenId** method (provided by the **%Persistent** class) uses this information to polymorphically open the correct type of object when multiple objects are stored within the same extent. This slot shows up in the class storage definition as a property named “%%CLASSNAME”.

For more details, refer to the section on [subclasses](#) below.

CAUTION: Globals that are part of an extent are managed by corresponding ObjectScript and SQL code. Any changes made to such a global through direct global access may corrupt the structure of the global (rendering its data inaccessible) or otherwise compromise access to its data through ObjectScript or SQL.

To prevent this, you should not use the [kill](#) command on globals for tasks like dropping all the data in an extent. Instead, you should use API methods such as **%KillExtent()** or [TRUNCATE TABLE](#), which perform important maintenance, such as resetting associated in-memory counters. However, classes that use a customized storage definition to project data from globals, are fully managed by application code, are exceptions to this rule. In such classes, you should consider setting either [READONLY](#) to 1 or [MAN-AGEDEXTENT](#) to 0 or both.

4.1.2 IDKEY

The IDKEY mechanism allows you to explicitly define the value used as an object ID. To do this, you simply add an IDKEY index definition to your class and specify the property or properties that will provide the ID value. Note that once you save an object, its object ID value cannot change. This means that after you save an object that uses the IDKEY mechanism, you can no longer modify any of the properties on which the object ID is based.

For example, we can modify the Person class used in the previous example to use an IDKEY index:

Class Definition

```
Class MyApp.Person Extends %Persistent
{
Index IDKEY On Name [ Idkey ];
Property Name As %String;
Property Age As %Integer;
}
```

If we create and save two instances of the Person class, the resulting global is now similar to:

```
^MyApp.PersonD("Abraham") = $LB("",530,"Abraham")
^MyApp.PersonD("Philip") = $LB("",680,"Philip")
```

Note that there is no longer any counter node defined. Also note that by basing the object ID on the Name property, we have implied that the value of Name must be unique for each object.

If the IDKEY index is based on multiple properties, then the main data nodes has multiple subscripts. For example:

Class Definition

```
Class MyApp.Person Extends %Persistent
{
Index IDKEY On (Name, Age) [ Idkey ];
Property Name As %String;
Property Age As %Integer;
}
```

In this case, the resulting global will now be similar to:

```
^MyApp.PersonD("Abraham", 530) = $LB("", 530, "Abraham")
^MyApp.PersonD("Philip", 680) = $LB("", 680, "Philip")
```

Important: There must not be a sequential pair of vertical bars (| |) within the values of any property used by an IDKEY index, unless that property is a valid reference to an instance of a persistent class. This restriction is imposed by the way in which the InterSystems SQL mechanism works. The use of | | in IDKey properties can result in unpredictable behavior.

4.1.3 Subclasses

By default, any fields introduced by a subclass of a persistent object are stored in an additional node. The name of the subclass is used as an additional subscript value.

For example, suppose we define a simple persistent MyApp.Person class with two literal properties:

Class Definition

```
Class MyApp.Person Extends %Persistent
{
Property Name As %String;
Property Age As %Integer;
}
```

Now we define a persistent subclass, MyApp.Student, that introduces two additional literal properties:

Class Definition

```
Class MyApp.Student Extends Person
{
Property Major As %String;
Property GPA As %Double;
}
```

If we create and save two instances of this MyApp.Student class, the resulting global will be similar to:

```
^MyApp.PersonD = 2 // counter node
^MyApp.PersonD(1) = $LB("Student", 19, "Jack")
^MyApp.PersonD(1, "Student") = $LB(3.2, "Physics")
^MyApp.PersonD(2) = $LB("Student", 20, "Jill")
^MyApp.PersonD(2, "Student") = $LB(3.8, "Chemistry")
```

The properties inherited from the Person class are stored in the main node, and those introduced by the Student class are stored in an additional subnode. This structure ensures that the Student data can be used interchangeably as Person data. For example, an SQL query listing names of all Person objects correctly picks up both Person and Student data. This structure also makes it easier for the Class Compiler to maintain data compatibility as properties are added to either the super- or subclasses.

Note that the first piece of the main node contains the string “Student” — this identifies nodes containing Student data.

4.1.4 Parent-Child Relationships

Within parent-child relationships, instances of child objects are stored as subnodes of the parent object to which they belong. This structure ensures that child instance data is physically clustered along with parent data.

For example, here is the definition for two related classes, Invoice:

Class Definition

```
/// An Invoice class
Class MyApp.Invoice Extends %Persistent
{
Property CustomerName As %String;

/// an Invoice has CHILDREN that are LineItems
Relationship Items As LineItem [inverse = TheInvoice, cardinality = CHILDREN];
}
```

and LineItem:

Class Definition

```
/// A LineItem class
Class MyApp.LineItem Extends %Persistent
{
Property Product As %String;
Property Quantity As %Integer;

/// a LineItem has a PARENT that is an Invoice
Relationship TheInvoice As Invoice [inverse = Items, cardinality = PARENT];
}
```

If we store several instances of Invoice object, each with associated LineItem objects, the resulting global will be similar to:

```
^MyApp.InvoiceD = 2 // invoice counter node
^MyApp.InvoiceD(1) = $LB("", "Wiley Coyote")
^MyApp.InvoiceD(1, "Items", 1) = $LB("", "Rocket Roller Skates", 2)
^MyApp.InvoiceD(1, "Items", 2) = $LB("", "Acme Magnet", 1)

^MyApp.InvoiceD(2) = $LB("", "Road Runner")
^MyApp.InvoiceD(2, "Items", 1) = $LB("", "Birdseed", 30)
```

For more information on relationships, see [Relationships](#).

4.1.5 Embedded Objects

Embedded objects are stored by first converting them to a serialized state (by default a **\$List** structure containing the object's properties) and then storing this serial state in the same way as any other property.

For example, suppose we define a simple serial (embeddable) class with two literal properties:

Class Definition

```
Class MyApp.MyAddress Extends %SerialObject
{
Property City As %String;
Property State As %String;
}
```

We now modify our earlier example to add an embedded Home address property:

Class Definition

```
Class MyApp.MyClass Extends %Persistent
{
Property Name As %String;
Property Age As %Integer;
Property Home As MyAddress;
}
```

If we create and save two instances of this class, the resulting global is equivalent to:

```
^MyApp.MyClassD = 2 // counter node
^MyApp.MyClassD(1) = $LB(530,"Abraham",$LB("UR","Mesopotamia"))
^MyApp.MyClassD(2) = $LB(680,"Philip",$LB("Bethsaida","Israel"))
```

4.1.6 Streams

Global streams are stored within globals by splitting their data into a series of chunks, each smaller than 32K bytes, and writing the chunks into a series of sequential nodes. File streams are stored in external files.

4.2 Indices

Persistent classes can define one or more indexes; additional data structures are used to make operations (such as sorting or conditional searches) more efficient. InterSystems SQL makes use of such indexes when executing queries. InterSystems IRIS Object and SQL automatically maintain the correct values within indexes as insert, update, and delete operations are carried out.

4.2.1 Storage Structure of Standard Indexes

A standard index associates an ordered set of one or more property values with the object ID values of the object containing the properties.

For example, suppose we define a simple persistent MyApp.Person class with two literal properties and an index on its Name property:

Class Definition

```
Class MyApp.Person Extends %Persistent
{
Index NameIdx On Name;

Property Name As %String;
Property Age As %Integer;
}
```

If we create and save several instances of this Person class, the resulting data and index globals is similar to:

```
// data global
^MyApp.PersonD = 3 // counter node
^MyApp.PersonD(1) = $LB("",34,"Jones")
^MyApp.PersonD(2) = $LB("",22,"Smith")
^MyApp.PersonD(3) = $LB("",45,"Jones")

// index global
^MyApp.PersonI("NameIdx"," JONES",1) = ""
^MyApp.PersonI("NameIdx"," JONES",3) = ""
^MyApp.PersonI("NameIdx"," SMITH",2) = ""
```

Note the following things about the index global:

1. By default, it is placed in a global whose name is the class name with an “I” (for Index) appended to it.
2. By default, the first subscript is the index name; this allows multiple indexes to be stored in the same global without conflict.
3. The second subscript contains the *collated* data value. In this case, the data is collated using the default SQLUPPER collation function. This converts all characters to uppercase (to sort without regard to case) and prepends a space character (to force all data to collate as strings).
4. The third subscript contains the Object ID value of the object that contains the indexed data value.
5. The nodes themselves are empty; all the needed data is held within the subscripts. Note that if an index definition specifies that data should be stored along with the index, it is placed in the nodes of the index global.

This index contains enough information to satisfy a number of queries, such as listing all Person class order by Name.

4.3 Bitmap Indexes

A bitmap index is similar to a standard index except that it uses a series of bitstrings to store the set of object ID values that correspond to the indexed value.

4.3.1 Logical Operation of Bitmap Indexes

A bitstring is a string containing a set of bits (0 and 1 values) in a special compressed format. InterSystems IRIS includes a set of functions to efficiently create and work with bitstrings:

- **\$Bit** — Set or get a bit within a bitstring. ‘
- **\$BitCount** — Count the number of bits within a bitstring.
- **\$BitFind** — Find the next occurrence of a bit within a bitstring.
- **\$BitLogic** — Perform logical (AND, OR) operations on two or more bitstrings.

Within a bitmap index, ordinal positions within a bitstring correspond to rows (Object ID number) within the indexed table. For a given value, a bitmap index maintains a bitstring that contains 1 for each row in which the given value is present, and contains 0 for every row in which it is absent. Note that bitmap indexes only work for objects that use the default storage structure with system-assigned, numeric Object ID values.

For example, suppose we have a table similar to the following:

ID	State	Product
1	MA	Hat
2	NY	Hat
3	NY	Chair
4	MA	Chair
5	MA	Hat

If the State and Product columns have bitmap indexes, then they contain the following values:

A bitmap index on the State column contains the following bitstring values:

MA	1	0	0	1	1
NY	0	1	1	0	0

Note that for the value, “MA”, there is a 1 in the positions (1, 4, and 5) that correspond to the table rows with State equal to “MA”.

Similarly, a bitmap index on the Product column contains the following bitstring values (note that the values are collated to uppercase within the index):

CHAIR	0	0	1	1	0
HAT	1	1	0	0	1

The InterSystems SQL Engine can execute a number of operations by iterating over, counting the bits within, or performing logical combinations (AND, OR) on the bitstrings maintained by these indexes. For example, to find all rows that have State equal to “MA” and Product equal to “HAT”, the SQL Engine can simply combine the appropriate bitstrings together with logical AND.

In addition to these indexes, the system maintains an additional index, called an “extent index,” that contains a 1 for every row that exists and a 0 for rows that do not (such as deleted rows). This is used for certain operations, such as negation.

4.3.2 Storage Structure of Bitmap Indexes

A bitmap index associates an ordered set of one or more property values with one or more bitstrings containing the Object ID values corresponding to the property values.

For example, suppose we define a simple persistent MyApp.Person class with two literal properties and a bitmap index on its Age property:

Class Definition

```
Class MyApp.Person Extends %Persistent
{
  Index AgeIdx On Age [Type = bitmap];

  Property Name As %String;
  Property Age As %Integer;
}
```

If we create and save several instances of this Person class, the resulting data and index globals is similar to:

```
// data global
^MyApp.PersonD = 3 // counter node
^MyApp.PersonD(1) = $LB("", 34, "Jones")
^MyApp.PersonD(2) = $LB("", 34, "Smith")
^MyApp.PersonD(3) = $LB("", 45, "Jones")

// index global
^MyApp.PersonI("AgeIdx", 34, 1) = 110...
^MyApp.PersonI("AgeIdx", 45, 1) = 001...

// extent index global
^MyApp.PersonI("$Person", 1) = 111...
^MyApp.PersonI("$Person", 2) = 111...
```

Note the following things about the index global:

1. By default, it is placed in a global whose name is the class name with an “I” (for Index) appended to it.
2. By default, the first subscript is the index name; this allows multiple indexes to be stored in the same global without conflict.

3. The second subscript contains the *collated* data value. In this case, a collation function is not applied as this is an index on numeric data.
4. The third subscript contains a *chunk* number; for efficiency, bitmap indexes are divided into a series of bitstrings each containing information for about 64000 rows from the table. Each of these bitstrings are referred to as a chunk.
5. The nodes contain the bitstrings.

Also note: because this table has a bitmap index, an extent index is automatically maintained. This extent index is stored within the index global and uses the class name, with a “\$” character prepended to it, as its first subscript.

4.3.3 Direct Access of Bitmap Indexes

The following example uses a class extent index to compute the total number of stored object instances (rows). Note that it uses [\\$Order](#) to iterate over the chunks of the extent index (each chunk contains information for about 64000 rows):

Class Member

```
/// Return the number of objects for this class.<BR>
/// Equivalent to SELECT COUNT(*) FROM Person
ClassMethod Count() As %Integer
{
    New total,chunk,data
    Set total = 0

    Set chunk = $Order(^MyApp.PersonI("$Person",""),1,data)
    While (chunk '= "") {
        Set total = total + $bitcount(data,1)
        Set chunk = $Order(^MyApp.PersonI("$Person",chunk),1,data)
    }

    Quit total
}
```

4.4 See Also

- [Introduction to Globals](#)
- [Introduction to Persistent Objects](#)
- [Persistent Objects and Storage Globals](#)

5

Temporary Globals and the IRISTEMP Database

For some operations, you may need the power of [globals](#) without requiring the data to be saved indefinitely. For example, you may want to use a global to sort some data which you do not need to store to disk. For these operations, InterSystems IRIS® data platform provides the mechanism of *temporary globals*.

Temporary globals have the following characteristics:

- Temporary globals are stored within the IRISTEMP database, which is always defined to be a local (that is, a non-network) database. All globals mapped to the IRISTEMP database are treated as temporary globals.
- Changes to temporary globals are not written to disk. Instead the changes are maintained within the in-memory buffer pool. A large temporary global may be written to disk if there is not sufficient space for it within the buffer pool.
- For maximum efficiency, changes to temporary globals are not logged to a journal file.
- Temporary globals are automatically deleted whenever InterSystems IRIS is restarted. (Note: it can be a very long time before a live system is restarted; so you should not count on this for cleaning up temporary globals.)

Tip: Temporary globals are useful when you need temporary data for use by multiple processes. If you need temporary data for use only within a single process, consider using a [process-private global](#), which is a special form of variable that is available only within the process that creates it and that is automatically removed when the process ends.

5.1 Using Temporary Globals

The mechanism for using temporary globals works as follows:

- For your application namespace, you define a global mapping so that globals with a specific naming convention are to be mapped to the IRISTEMP database, which is a special database as discussed below.

For example, you might define a global mapping so that all globals with names of the form ^AcmeTemp* are mapped to the IRISTEMP database.
- When your code needs to store data temporarily and read it again, your code writes to and reads from globals that use that naming convention.

By using temporary globals, you take advantage of the fact that the IRISTEMP database is not [journaled](#). Because the database is not journaled, operations that use the database do not result in journal files. Journal files can become large and can cause space issues. However, note the following points:

- You cannot roll back any transactions that modify globals in the IRISTEMP database; this behavior is specific to IRISTEMP. If you need to manage temporary work via transactions, do not use globals in IRISTEMP for that purpose.
- Take care to use IRISTEMP only for work that does not need to be saved.
- The IRISTEMP database increases in size when it requires more memory. You can use the [MaxIRISTempSizeAtStart](#) parameter to help manage the size of IRISTEMP.

5.2 Defining a Mapping for Temporary Globals

To define a mapping for temporary globals, do the following:

1. Choose a naming convention and ensure that all of your developers are aware of it. Note the following points:
 - Consider whether to have many temporary globals or fewer temporary globals with multiple nodes. It is easier for InterSystems IRIS to efficiently read or write different nodes within the same global, compared to reading or writing the equivalent number of separate globals. The efficiency difference is negligible for small numbers of globals but is noticeable when there are hundreds of separate globals.
 - If you plan to use the same global mapping in multiple namespaces, then devise a system so that work in one namespace does not interfere with work in another namespace. For example, you could use the namespace name as a subscript in the globals.
 - Similarly, even within one namespace, devise a system so that each part of the code uses a different global or a different subscript in the same global, again to avoid interference.
 - Do not use system-reserved global names. See [Global Variable Names to Avoid](#).
2. In the Management Portal, navigate to the **Namespaces** page (**System Administration > Configuration > System Configuration > Namespaces**).
3. In the row for your application namespace, click **Global Mappings**.
4. From the **Global Mappings** page, click **New Global Mapping**.
5. For **Global database location**, select IRISTEMP.
6. For **Global name**, enter a name ending in an asterisk (*). Do not include the initial caret of the name.
For example: AcmeTemp*
This mapping causes all globals with names that start AcmeTemp* to be mapped to the IRISTEMP database.
7. Click **OK**.
Note: The >> symbol displayed in the first column of the new mappings row indicates that you opened the mapping for editing.
8. To save the mappings so that InterSystems IRIS uses them, click **Save Changes**.

For more details, see [Configuring Namespaces](#).

5.3 System Use of IRISTEMP

Note that InterSystems uses temporary system globals as scratch space, for example, as temporary indexes during the execution of certain queries (for sorting, grouping, calculating aggregates, etc.). These globals are automatically mapped to IRISTEMP and include:

- `^IRIS.Temp*`
- `^CacheTemp*`
- `^mtemp*`

Never change any of these globals.

5.4 ^CacheTemp Globals

Historically, customers have used globals having names starting with `^CacheTemp` as temporary globals. By convention, these globals use names starting with `^CacheTempUser` to avoid possible conflict with temporary system globals. However, the best practice is to define your own temporary globals and map them to IRISTEMP, as described in [Using Temporary Globals](#).

5.5 See Also

- [Introduction to Globals](#)
- [Configuring Namespaces](#)
- [Process-Private Globals](#)
- [Using Temporary Globals](#)

